Práctica: Colas en Java (Estructuras de Datos)

Dato	Detalle
Asignatura	Estructuras de Datos
Duración estimada	2 horas (120 min)
Modalidad	Individual
Nombre del estudiante	Molina Rosas Maria Valeria
Fecha	30/octubre/2026

1. Objetivos de aprendizaje

Al finalizar esta práctica, el estudiante será capaz de:

1. Explicar el principio FIFO y las operaciones básicas de una cola.
2. Implementar una cola con arreglo circular.
3. Implementar una cola con nodos enlazados usando genéricos.
4. Utilizar la interfaz `Queue` de la librería estándar de Java.
5. Aplicar colas para resolver un problema de simulación.

2. Requisitos previos

- Conocimientos básicos de Java: clases, objetos, arreglos, ciclos y excepciones.
- Haber visto el concepto de nodo y referencias (para la Parte 2).
- JDK 11 o superior instalado.
- VS Code con la extensión de Java.

3. Resumen teórico

Una cola es una estructura lineal que sigue el principio FIFO (First In, First Out): el primer elemento en entrar es el primero en salir.

Operación	Descripción	Complejidad
encolar	Agrega un elemento al final	O(1)
desencolar	Elimina y devuelve el elemento del frente	O(1)
frente	Consulta el frente sin eliminarlo	O(1)
estaVacia	Indica si no hay elementos	O(1)
tamanio	Devuelve la cantidad de elementos	O(1)

```
Frente                  Final
  ↓                       ↓
```

```
[ 10 | 20 | 30 | 40 ]  ← encolar(50)
  ↑
desencolar() → sale el 10
```

4. Preparación del entorno

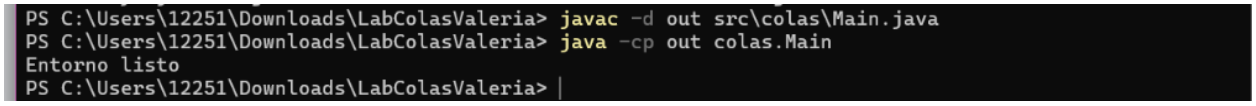
1. **Paso 1.** Crea un nuevo proyecto Java llamado `LabColas_[nombre]`.
2. **Paso 2.** Dentro de `src`, crea el paquete `colas`.
3. **Paso 3.** Verifica que todo funciona creando la clase `Main` con este código y ejecutándola:

```
package colas;

public class Main {
    public static void main(String[] args) {
        System.out.println("Entorno listo");
    }
}
```

Punto de control: la consola muestra `Entorno listo`.

[PEGA TU IMAGEN ACÁ]



Parte 1. Cola con arreglo circular

En un arreglo normal, al desencolar quedan espacios vacíos al inicio que no se reutilizan. En un arreglo circular, los índices "dan la vuelta" usando el operador módulo (`%`).

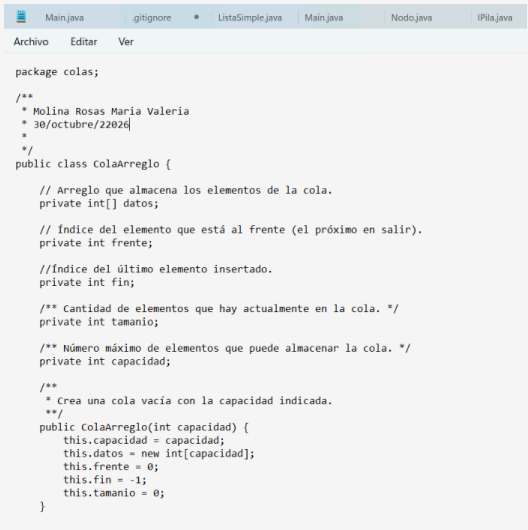
Paso 1.1. Crear la clase y sus atributos

Crea la clase `ColaArreglo` en el paquete `colas` y documenta con comentarios la descripción de la clase, sus propiedades y sus métodos.

```
package colas;

public class ColaArreglo {
    private int[] datos;
    private int frente;
    private int fin;
    private int tamanio;
    private int capacidad;

    public ColaArreglo(int capacidad) {
        this.capacidad = capacidad;
        this.datos = new int[capacidad];
        this.frente = 0;
        this.fin = -1;
        this.tamanio = 0;
    }
}
```



Paso 1.2. Métodos de consulta

Agrega dentro de la clase:

```
public boolean estaVacia() {
    return tamanio == 0;
}

public boolean estaLlena() {
    return tamanio == capacidad;
}

public int tamanio() {
    return tamanio;
}
```

```
    public boolean estaVacia() {
        return tamanio == 0;
    }

    /**
     * Indica si la cola alcanzó su capacidad máxima.
     *
     * @return true si está llena
     */
    public boolean estaLlena() {
        return tamanio == capacidad;
    }

    /**
     * Devuelve la cantidad de elementos de la cola.
     *
     * @return número de elementos
     */
    public int tamanio() {
        return tamanio;
    }

    /**
     * Agrega un elemento al final de la cola. Complejidad O(1).
     *
     * @param valor valor a encolar
     * @throws IllegalStateException si la cola está llena
     */
    public void encolar(int valor) {
        if (estaLlena()) {
            throw new IllegalStateException("La cola está llena");
        }
        fin = (fin + 1) % capacidad; // avance circular
        datos[fin] = valor;
        tamanio++;
    }
```

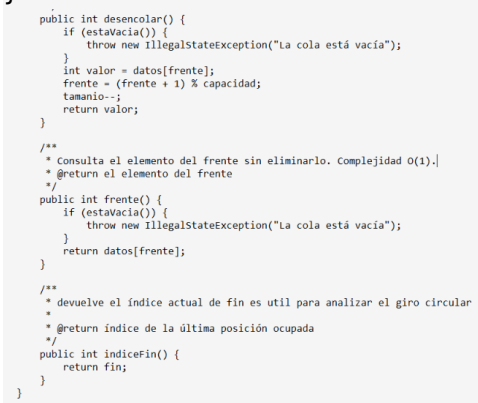
Paso 1.3. Método encolar

```
public void encolar(int valor) {
    if (estaLlena()) {
        throw new IllegalStateException("La cola está llena");
    }
    fin = (fin + 1) % capacidad; // avance circular
    datos[fin] = valor;
    tamanio++;
}
```

Paso 1.4. Métodos desencolar y frente

```
public int desencolar() {
    if (estaVacia()) {
        throw new IllegalStateException("La cola está vacía");
    }
    int valor = datos[frente];
    frente = (frente + 1) % capacidad;
    tamanio--;
    return valor;
}

public int frente() {
    if (estaVacia()) {
        throw new IllegalStateException("La cola está vacía");
    }
    return datos[frente];
}
```



```
    public int desencolar() {
        if (estaVacia()) {
            throw new IllegalStateException("La cola está vacía");
        }
        int valor = datos[frente];
        frente = (frente + 1) % capacidad;
        tamanio--;
        return valor;
    }

    /**
     * Consulta el elemento del frente sin eliminarlo. Complejidad O(1).
     * @return el elemento del frente
     */
    public int frente() {
        if (estaVacia()) {
            throw new IllegalStateException("La cola está vacía");
        }
        return datos[frente];
    }

    /**
     * devuelve el índice actual de fin es util para analizar el giro circular
     *
     * @return índice de la última posición ocupada
     */
    public int indiceFin() {
        return fin;
    }
}
```

Paso 1.5. Probar la cola

En `Main`, escribe esta prueba que fuerza el "giro" del arreglo:

```
ColaArreglo cola = new ColaArreglo(3);
cola.encolar(10);
cola.encolar(20);
cola.encolar(30);
System.out.println("Sale: " + cola.desencolar());   // 10
cola.encolar(40);                                    // reutiliza la posición 0
System.out.println("Frente: " + cola.frente());     // 20
System.out.println("Tamaño: " + cola.tamanio());    // 3
System.out.println("¿Llena? " + cola.estaLlena());  // true
```

Punto de control: la salida debe ser `Sale: 10`, `Frente: 20`, `Tamaño: 3`, `¿Llena? True`.

```java
package colas;

/**
 * Clase principal del laboratorio de colas.
 * Verifica el entorno y ejecuta las pruebas de la Parte 1 (cola circular)
 * y la Parte 2 (cola enlazada genérica).
 */
public class Main {

    /**
     * Punto de entrada del programa.
     *
     * @param args argumentos de línea de comandos (no se usan)
     */
    public static void main(String[] args) {
        System.out.println("Entorno listo");

        ColaArreglo cola = new ColaArreglo(3);
        cola.encolar(10);
        cola.encolar(20);
        cola.encolar(30);
        System.out.println("Sale: " + cola.desencolar());   // 10
        cola.encolar(40);                                   // reutiliza la posición 0
        System.out.println("Frente: " + cola.frente());     // 20
        System.out.println("Tamaño: " + cola.tamanio());    // 3
        System.out.println("¿Llena? " + cola.estaLlena());  // true
```

Preguntas de análisis (Parte 1)

1. ¿Qué valor tiene fin después de encolar 40? Vale 0. Con capacidad 3, fin estaba en 2; al calcular (2 + 1) % 3 el índice da la vuelta y vale 0, la posición que se liberó al desencolar el 10.

2. ¿Qué pasaría sin el módulo? fin pasaría a 3 y se intentaría escribir fuera del arreglo (ArrayIndexOutOfBoundsException), aunque hubiera espacio libre al inicio.

3. ¿Qué ocurre al encolar con la cola llena? Se lanza IllegalStateException("La cola está llena"). Una versión dinámica crearía un arreglo del doble de tamaño, copiaría los elementos en orden desde frente y reajustaría frente y fin.

Parte 2. Cola con nodos enlazados

Idea clave: esta cola no tiene límite de capacidad. Se mantienen dos referencias: `frente` (donde se elimina) y `fin` (donde se inserta).

Paso 2.1. Crear la clase con el nodo

Es necesario documentar cada clase, propiedad y método.

```java
package colas;

public class Nodo<T> {
    T dato;
    Nodo<T> siguiente;

    Nodo(T dato) {
        this.dato = dato;
    }
}
```

```java
package colas;

/**
 * Molina Rosas Maria Valeria
 *30/octubre/2026
 */
public class Nodo<T> {

    /** Dato almacenado en el nodo. */
    T dato;

    /** Referencia al siguiente nodo null si es el ultimo */
    Nodo<T> siguiente;

    /**
     * Crea un nodo con el dato indicado y sin siguiente.
     *
     * @param dato valor que guardará el nodo
     */
    Nodo(T dato) {
        this.dato = dato;
    }
}
```

```java
package colas;

public class ColaEnlazada<T> {
    private Nodo<T> frente;
    private Nodo<T> fin;
    private int tamanio;
}
```

```java
package colas;

/**
 * Molina Rosas Maria Valeria
 * 30/octubre/2026
 **/
public class ColaEnlazada<T> {

    /** Primer nodo de la cola (el próximo en salir). */
    private Nodo<T> frente;

    /** Último nodo de la cola (donde se inserta). */
    private Nodo<T> fin;

    /** Cantidad de elementos en la cola. */
    private int tamanio;
```

Paso 2.2. Métodos estaVacia y tamanio

```java
public boolean estaVacia() {
    return frente == null;
}

public int tamanio() {
    return tamanio;
}
```

```
    public boolean estaVacia() {
        return frente == null;
    }

    /**
     * Devuelve la cantidad de elementos de la cola.
     *
     * @return número de elementos
     */
    public int tamanio() {
        return tamanio;
    }
```

Paso 2.3. Método encolar

Piensa en los dos casos: cola vacía y cola con elementos.

```
public void encolar(T valor) {
    Nodo<T> nuevo = new Nodo<>(valor);
    if (estaVacia()) {
        frente = nuevo;
    } else {
        fin.siguiente = nuevo;
    }
    fin = nuevo;
    tamanio++;
}
```

```
    //Elimina y devuelve el elemento del frente. Complejidad O(1).
    public T desencolar() {
        if (estaVacia()) {
            throw new IllegalStateException("La cola está vacía");
        }
        T valor = frente.dato;
        frente = frente.siguiente;
        if (frente == null) {
            fin = null; // la cola quedó vacía
        }
        tamanio--;
        return valor;
    }
```

Paso 2.4. Método desencolar

```
public T desencolar() {
    if (estaVacia()) {
        throw new IllegalStateException("La cola está vacía");
    }
    T valor = frente.dato;
    frente = frente.siguiente;
    if (frente == null) {
        fin = null; // la cola quedó vacía
    }
    tamanio--;
    return valor;
}
```

Paso 2.5. Método frente

```
public T frente() {
    if (estaVacia()) {
        throw new IllegalStateException("La cola está vacía");
```

```
    }
    return frente.dato;
}
```

```
    // Consulta el elemento del frente sin eliminarlo. Complejidad O(1).

    public T frente() {
        if (estaVacia()) {
            throw new IllegalStateException("La cola está vacía");
        }
        return frente.dato;
    }
```

Paso 2.6. Probar con genéricos

```
ColaEnlazada<String> nombres = new ColaEnlazada<>();
nombres.encolar("Ana");
nombres.encolar("Luis");
nombres.encolar("María");
System.out.println(nombres.desencolar()); // Ana
System.out.println(nombres.frente());     // Luis
System.out.println(nombres.tamanio());    // 2
```

Punto de control: la salida debe ser `Ana`, `Luis`, `2`.

Preguntas de análisis (Parte 2)

1. ¿Por qué fin = null al quedar vacía? Si frente es null pero fin sigue apuntando al último nodo, queda una referencia obsoleta nodo fantasma; en la siguiente inserción se podría enlazar a un nodo ya eliminado y se mantiene memoria sin liberar.

2. Ventaja y costo frente al arreglo. No tiene capacidad fija ni desperdicia espacio. El costo es memoria extra: cada elemento lleva una referencia a su siguiente nodo y el objeto Nodo.

3. ¿Por qué T en lugar de int? Los genéricos permiten reutilizar la misma cola para cualquier tipo String, Integer, objetos propios con verificación de tipos en compilación y sin duplicar código.

Parte 3. La interfaz Queue de Java (15 min)

Paso 3.1. Usar Queue con LinkedList y ArrayDeque

```
import java.util.ArrayDeque;
import java.util.LinkedList;
import java.util.Queue;

Queue<String> cola = new LinkedList<>();
// Prueba también: Queue<String> cola = new ArrayDeque<>();
cola.offer("Ana");
cola.offer("Luis");
cola.offer("María");
System.out.println(cola.peek());  // Ana (no la elimina)
System.out.println(cola.poll());  // Ana (la elimina)
System.out.println(cola.size());  // 2
System.out.println(cola);         // [Luis, María].
```

```java
import java.util.ArrayDeque;
import java.util.LinkedList;
import java.util.NoSuchElementException;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * Molina Rosas Maria Valeria
 *30/octubre/2026
 */
public class PruebaQueue {

    /**
     * Ejecuta las pruebas de la Parte 3.
     */
    public static void main(String[] args) {
        Queue<String> cola = new LinkedList<>();
        // Prueba también: Queue<String> cola = new ArrayDeque<>();
        cola.offer("Ana");
        cola.offer("Luis");
        cola.offer("María");
        System.out.println(cola.peek()); // Ana (no la elimina)
        System.out.println(cola.poll()); // Ana (la elimina)
        System.out.println(cola.size()); // 2
        System.out.println(cola);        // [Luis, María]

        Queue<String> cola2 = new ArrayDeque<>();
        cola2.offer("Ana");
        cola2.offer("Luis");
        cola2.offer("María");
        System.out.println(cola2.peek());
        System.out.println(cola2.poll());
        System.out.println(cola2.size());
        System.out.println(cola2);

        Queue<String> vacia = new LinkedList<>();

        try {
            vacia.remove();
            System.out.println("remove(): sin error");
```

Paso 3.2. Comparar métodos con excepción vs. valor especial

Con una cola vacía, ejecuta cada método y completa la tabla:

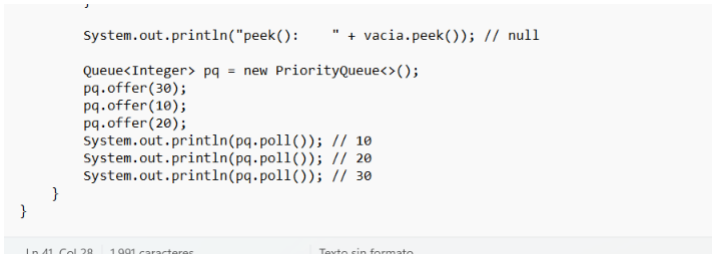
Método	¿Qué ocurrió? (excepción / null)
`remove()`	Lanzo NoSuchElementException
`poll()`	Devuelve null
`element()`	Lanzo NoSuchelementExeception
`peek()`	Devuelve null

Pista: los métodos `add`, `remove` y `element` lanzan excepción; `offer`, `poll` y `peek` devuelven un valor especial.

Paso 3.3. Cola de prioridad

```java
Queue<Integer> pq = new java.util.PriorityQueue<>();
pq.offer(30);
pq.offer(10);
pq.offer(20);
System.out.println(pq.poll()); // ¿Qué valor sale primero?
System.out.println(pq.poll());
System.out.println(pq.poll());
```

Pregunta: ¿en qué se diferencia el orden de salida de una `PriorityQueue` respecto a una cola FIFO normal?



```
        System.out.println("peek():    " + vacia.peek()); // null

        Queue<Integer> pq = new PriorityQueue<>();
        pq.offer(30);
        pq.offer(10);
        pq.offer(20);
        System.out.println(pq.poll()); // 10
        System.out.println(pq.poll()); // 20
        System.out.println(pq.poll()); // 30
    }
}
```

Ln 41, Col 28 | 1.991 caracteres | Texto sin formato

Parte 4. Problema integrador: simulación de fila de banco (25 min)

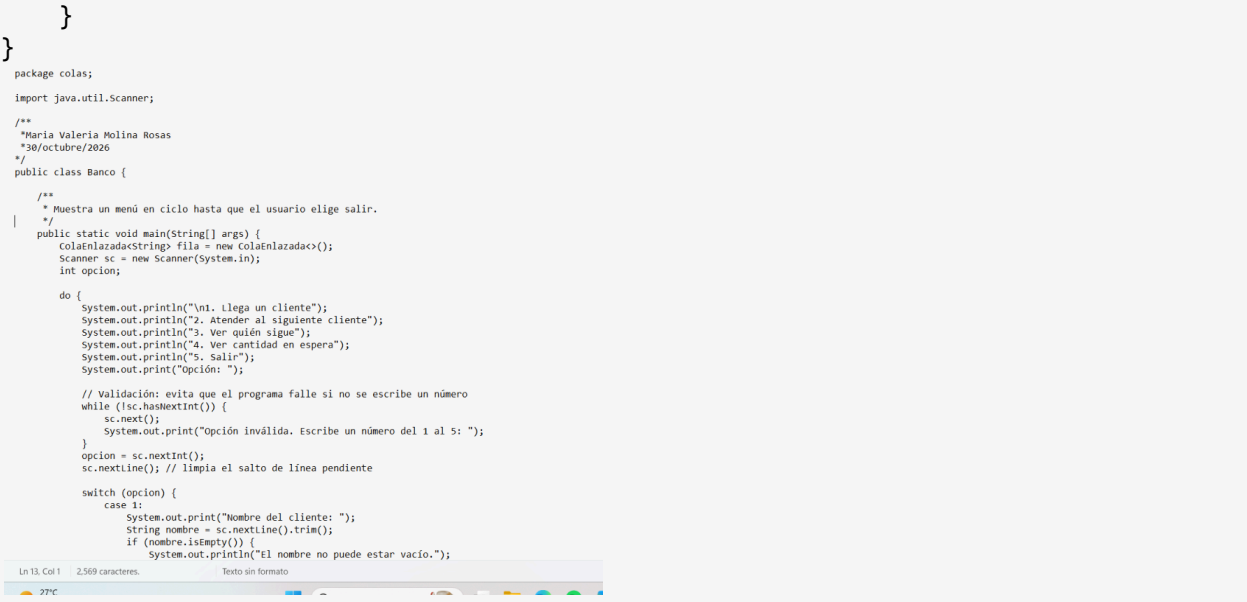
Enunciado: simula una fila de banco con un solo cajero usando tu clase `ColaEnlazada<String>`. El programa debe mostrar este menú en un ciclo hasta que el usuario elija salir:

1. Llega un cliente
2. Atender al siguiente cliente
3. Ver quién sigue
4. Ver cantidad de clientes en espera
5. Salir

Paso 4.1. Estructura base

```
import java.util.Scanner;

public class Banco {
    public static void main(String[] args) {
        ColaEnlazada<String> fila = new ColaEnlazada<>();
        Scanner sc = new Scanner(System.in);
        int opcion;
        do {
            System.out.println("\n1. Llega un cliente");
            System.out.println("2. Atender al siguiente cliente");
            System.out.println("3. Ver quién sigue");
            System.out.println("4. Ver cantidad en espera");
            System.out.println("5. Salir");
            System.out.print("Opción: ");
            opcion = sc.nextInt();
            sc.nextLine();
            switch (opcion) {
                case 1:
                    // TODO: pedir el nombre y encolarlo
                    break;
                case 2:
                    // TODO: desencolar (validar que no esté vacía)
                    break;
                case 3:
                    // TODO: mostrar el frente (validar que no esté vacía)
                    break;
                case 4:
                    // TODO: mostrar el tamaño
                    break;
            }
        } while (opcion != 5);
```



```
    }
}
package colas;

import java.util.Scanner;

/**
 *Maria Valeria Molina Rosas
 *30/octubre/2026
*/
public class Banco {

    /**
     * Muestra un menú en ciclo hasta que el usuario elige salir.
     */
    public static void main(String[] args) {
        ColaEnlazada<String> fila = new ColaEnlazada<>();
        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("\n1. Llega un cliente");
            System.out.println("2. Atender al siguiente cliente");
            System.out.println("3. Ver quién sigue");
            System.out.println("4. Ver cantidad en espera");
            System.out.println("5. Salir");
            System.out.print("Opción: ");

            // Validación: evita que el programa falle si no se escribe un número
            while (!sc.hasNextInt()) {
                sc.next();
                System.out.print("Opción inválida. Escribe un número del 1 al 5: ");
            }
            opcion = sc.nextInt();
            sc.nextLine(); // limpia el salto de línea pendiente

            switch (opcion) {
                case 1:
                    System.out.print("Nombre del cliente: ");
                    String nombre = sc.nextLine().trim();
                    if (nombre.isEmpty()) {
                        System.out.println("El nombre no puede estar vacío.");
```

Ln 13, Col 1 | 2.569 caracteres. | Texto sin formato

Paso 4.2. Completar los TODO

Implementa cada caso. **Importante:** antes de atender o consultar, verifica `estaVacia()` e imprime un mensaje amigable en lugar de dejar que el programa falle con una excepción.

Paso 4.3. Casos de prueba

Ejecuta esta secuencia y verifica el resultado esperado:

Acción	Resultado esperado
Opción 2 con fila vacía	Mensaje "No hay clientes en espera"
Llegan Ana, Luis y María	Tamaño = 3
Opción 3	Muestra "Ana"
Opción 2 (dos veces)	Atiende a Ana y luego a Luis
Opción 4	Tamaño = 1

Punto de control: muestra tu programa funcionando al docente.

Parte 5. Ejercicios de refuerzo (según el tiempo disponible)

Elige al menos dos:

1. **Nivel básico:** agrega a `ColaEnlazada` un método `imprimirCola()` que muestre todos los elementos del frente al final, sin destruir la cola.
2. **Nivel básico:** agrega un método `vaciar()` que deje la cola sin elementos.
3. **Nivel intermedio:** invierte una cola usando una pila (`java.util.Stack` o `ArrayDeque` como pila).
4. **Nivel intermedio:** implementa una cola usando dos pilas.

5. **Nivel avanzado:** usa una cola para recorrer un árbol binario por niveles (BFS).

6. Entregables

Sube al GitHub:

- ☐ Código fuente: `ColaArreglo.java`, `ColaEnlazada.java`, `Banco.java` y `Main.java`.
- ☐ Código de los ejercicios de refuerzo elegidos.
- ☐ Un documento breve (máx. 1 página) con:
    - Las respuestas a las preguntas de análisis.
    - La tabla completada del Paso 3.2.
    - Capturas de pantalla de las pruebas ejecutadas.

7. Rúbrica de evaluación

Criterio	Puntos
`ColaArreglo` funciona correctamente (incluido el giro circular)	20
`ColaEnlazada` genérica funciona correctamente	25
Uso correcto de `Queue` y tabla del Paso 3.2 completa	15
Simulación del banco con validaciones	20
Ejercicios de refuerzo	10
Respuestas de análisis, orden y claridad del código	10
Total	**100**

8. Errores comunes a evitar

- Olvidar `% capacidad` al avanzar los índices en la cola circular.
- No actualizar `fin = null` al vaciar la cola enlazada.
- No validar `estaVacia()` antes de desencolar o `frente`.
- Confundir `peek()` (consulta) con `poll()` (elimina).
- Olvidar `sc.nextLine()` después de `nextInt()` al leer texto con `Scanner`.

9. Cierre y reflexión

Responde brevemente:

1. ¿En qué situaciones elegirías una cola con arreglo y en cuáles una enlazada?

Elegiría una cola con arreglo circular cuando conozco o puedo acotar el tamaño máximo de antemano, por ejemplo en un buffer de teclado o en una cola de impresión con límite fijo. Es más eficiente en memoria porque los datos están contiguos y no necesita un nodo extra por elemento. Elegiría una cola enlazada cuando el número de elementos es impredecible, como en la fila de un banco o en las peticiones de un servidor, porque no tiene capacidad máxima y nunca se llena. Su desventaja es el costo de memoria adicional de la referencia siguiente en cada nodo.

2. Menciona dos ejemplos de la vida real o de sistemas informáticos donde se usen colas.

Un ejemplo es la cola de impresión, donde los documentos se imprimen en el mismo orden en que llegaron. Otro es la atención de peticiones en un servidor web, donde las solicitudes de los usuarios esperan su turno para ser procesadas. También se usan en la fila de turnos de un banco.

3. ¿Qué fue lo más difícil de la práctica y cómo lo resolviste?

Lo más difícil fue entender el giro circular del arreglo, porque al principio no veía cómo fin podía volver a la posición 0. Lo resolví haciendo una prueba de escritorio con una cola de capacidad 3 y anotando los valores de frente, fin y tamanio después de cada operación; así comprendí que (fin + 1) % capacidad hace que el índice regrese al inicio. También me costó recordar asignar fin = null al vaciar la cola enlazada, y lo corregí probando el caso de desencolar hasta dejarla vacía.